4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques

OUTPUT

```
kannamina_abhishek@LAPTOP-D9F9IT80:~$ nano p4_wait.c
kannamina_abhishek@LAPTOP-D9F9IT80:~$ gcc p4_wait.c -o p4_wait
kannamina_abhishek@LAPTOP-D9F9IT80:~$ ./p4_wait
Parent PID = 506
Child 1: PID = 507
Child 2: PID = 508
wait() collected child PID = 507
waitpid() collected child PID = 508
kannamina_abhishek@LAPTOP-D9F9IT80:~$
```

***CODE*:**

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t c1, c2;

    c1 = fork();

    if (c1 == 0) {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        return 10;
    }

    c2 = fork();

    if (c2 == 0) {
        printf("Child 2: PID = %d\n", getpid());
        sleep(4);
        return 20;
    }

    printf("Parent PID = %d\n", getpid());

    int status;
    pid_t finished = wait(&status);

    printf("wait() collected child PID = %d\n", finished);

    waitpid(c2, &status, 0);

    printf("waitpid() collected child PID = %d\n", c2);

    return 0;
}
```